

Aula 7 — Projeto de um Analisador Léxico

Compiladores I

FACOM - Faculdade de Computação

Bloco 3: Análise Léxica

Duração estimada: 70–80 minutos

Revisão da Aula 6

- ▶ AFD x AFN; construção de Thompson e de subconjuntos;
- ▶ Minimização de AFD.

Pergunta rápida

O scanner de um compilador real, como o do Micro C, é implementado como um AFD ou como um AFN? Por quê?

Objetivos de aprendizagem

- ▶ Projetar a estrutura geral de um analisador léxico;
- ▶ Especificar o conjunto de tokens de uma linguagem simples;
- ▶ Distinguir token, padrão e lexema (revisão aplicada).

Sumário da aula

Estrutura de um analisador léxico

Ferramentas e especificação

Os tokens do Micro C: tokens.h

Exemplo — Vocabulário completo (Cap. 2.3.1)

```
1  typedef enum {
2      TK_INT, TK_FLOAT, TK_ID, TK_NUM,
3      TK_PLUS, TK_MINUS, TK_TIMES, TK_DIV,
4      TK_ASSIGN, TK_EQ, TK_SEMI,
5      TK_LPAREN, TK_RPAREN, TK_IF, TK_ELSE,
6      TK_EOF, TK_ERROR
7  } TokenType;
```

Cada valor do enum corresponde a uma categoria de lexema definida por uma expressão regular (Aula 4).

Atributos: o valor por trás do token

Por que token sozinho não basta

Saber que um token é TK_ID não diz *qual* identificador foi lido. É preciso um **atributo**.

Exemplo — Estrutura de um token com atributo

```
1 typedef struct {  
2     TokenType tipo;  
3     char lexema[64];  
4     int valor_num;    /* usado quando tipo == TK_NUM */  
5 } Token;
```

Tratamento de erros léxicos

- ▶ Caracteres não reconhecidos por nenhuma expressão regular (ex: @, \$) geram `TK_ERROR`;
- ▶ Boas práticas: reportar linha e coluna do erro, e tentar continuar a análise (não travar no primeiro erro);
- ▶ Palavras reservadas (`if`, `int`, ...) são identificadores que "colidem" com uma tabela de palavras-chave — resolvido após reconhecer o padrão de identificador.

Pergunta rápida

Como o scanner decide se `if` é a palavra reservada `TK_IF` ou um identificador comum?

Sumário da aula

Estrutura de um analisador léxico

Ferramentas e especificação

Scanner manual x gerador de scanners

Manual (Micro C)

- ▶ AFD implementado "à mão" em C, com switch/laços;
- ▶ Total controle e transparência didática;
- ▶ Mais trabalho de implementação.

Gerador (ex: Lex/Flex)

- ▶ Você escreve as expressões regulares;
- ▶ A ferramenta gera o AFD e o código do scanner automaticamente;
- ▶ Mais produtivo em compiladores reais.

Especificando tokens para um scanner

Exemplo — Especificação (estilo Lex, conceitual)

```
1 "if"                { retorna TK_IF }
2 [a-zA-Z_][a-zA-Z0-9_]* { retorna TK_ID }
3 [0-9]+              { retorna TK_NUM }
4 "="                 { retorna TK_ASSIGN }
5 "=="                { retorna TK_EQ }
6 [ \t\n]+            { ignora }
```

A ordem importa: padrões mais específicos (como palavras reservadas) geralmente têm prioridade sobre padrões gerais (como identificador).

Desafio em sala

Desafio em sala

Projete (em uma tabela ou lista) a especificação de tokens necessária para um mini-scanner que reconheça: palavras reservadas `int` e `return`, identificadores, números inteiros, os operadores `+`, `-`, `=` e o delimitador `;`. Indique o token gerado para cada padrão.

Síntese da aula

- ▶ Um analisador léxico real precisa de: enumeração de tokens, estrutura de atributos e tratamento de erros;
- ▶ Pode ser implementado manualmente (Micro C) ou via geradores (Lex/Flex);
- ▶ A especificação de tokens define, para cada padrão, qual token deve ser gerado.

Próxima aula: implementação prática de um analisador léxico (laboratório).

Teste rápido — Aula 7

Questões teóricas

1. Por que um token, sozinho, muitas vezes não é suficiente e precisa de um atributo?
2. Qual a vantagem de usar um gerador de scanners (como Flex) em vez de escrever o scanner manualmente?

Questão prática

3. Adicione ao enum `TokenType` do Micro C (slide 4) os tokens necessários para suportar o operador `%` (módulo) e a palavra reservada `while`.